

Risk Analysis and Management

Risk

- Risk is a **potential problem**. It's an activity or event that may compromise the success of a software development project.

Reactive Risk Management

- Project team reacts to risks when they occur.
- More commonly, the software team does nothing about risks until something goes wrong.
- Then, the team involved into action in an attempt to correct the problem rapidly. This is often called a *fire fighting mode*.
- When this fails, “crisis management” takes over and the project is in real jeopardy.

Proactive Risk Management

- A proactive strategy begins long before technical work is initiated.
- Potential risks are identified, their probability and impact are assessed, and they are ranked by importance.
- Then, the software team establishes a plan for managing risk.
- The primary objective is to avoid risk, but because not all risks can be avoided, the team works to develop a contingency plan that will enable it to respond in a controlled and effective manner.

Software Risks

- Risk always involves two characteristics:
 - **Uncertainty** —the risk may or may not happen; that is, there are no 100% probable risks.
 - **Loss**—if the risk becomes a reality, unwanted consequences or losses will occur
- When risks are analyzed, it is important to quantify the level of uncertainty and the degree of loss associated with each risk.
- To accomplish this, different categories or types of risks are considered.
 - Project Risks
 - Technical Risks
 - Business Risks
 - Known Risks.
 - Predictable Risks
 - Unpredictable Risks

Project Risk

- *Project risks* make threats the **project plan**.
- That is, if project risks become real, it is likely that project schedule will slip and that costs will increase.
- Project risks identify potential budgetary, schedule, personnel (staffing and organization), resource, customer, and requirements problems and their impact on a software project.
- Project complexity, size, and the degree of structural uncertainty were also defined as project risk factors.

Technical risks

- *Technical risks* threaten the **quality and timeliness** of the software to be produced.
- If a technical risk becomes a reality, implementation may become difficult or impossible.
- Technical risks identify potential design, implementation, interface, verification, and maintenance problems.
- In addition, specification ambiguity, technical uncertainty, and "leading-edge" technology are also risk factors.

Business Risk

- *Business risks* threaten the **feasibility** of the software to be built.
- Business risks often jeopardize the project or the product.
- Top five business risks are
 - Building a excellent product or system that no one really wants (market risk),
 - Building a product that no longer fits into the overall business strategy for the company (strategic risk)
 - Building a product that the sales force doesn't understand how to sell.
 - Losing the support of senior management due to a change in focus or a change in people (management risk)
 - Losing budgetary or personnel commitment (budget risks)

Known Risk

- *Known risks* are those that can be **uncovered after careful evaluation of the project plan**, the business and technical environment in which the project is being developed,
- Other reliable information sources (e.g., unrealistic delivery date, lack of documented requirements or software scope, poor development environment).

- ***Predictable risks*** are generalized from **past project experience** (e.g., staff turnover, poor communication with the customer, etc).
- ***Unpredictable risks*** are the joker in the deck. They can and do occur, but they are extremely **difficult to identify** in advance.

Risk Identification

- ***Risk identification is a systematic attempt to specify threats to the project plan (estimates, schedule, resource loading, etc.).***
- By identifying known and predictable risks, the project manager takes a first step toward avoiding them when possible and controlling them when necessary.
- Two distinct types of risks
 - ***Generic risks*** are a potential threat to every software project
 - ***Product-specific risks*** can be identified only by those with a clear understanding of the technology, the people, and the environment that is specific to the project at hand.

Contd.

- To identify product-specific risks, the project Plan and the software statement of scope are examined.
- One method for identifying risks is to create a ***risk item checklist***.
- The checklist can be used for risk identification and focuses on some subset of *known and predictable risks* in the following generic subcategories:
 - ***Product size*** —risks associated with the overall size of the software to be built or modified.
 - ***Business impact*** —risks associated with constraints imposed by management or the marketplace.

Risk Item Checklist contd.

- ***Customer characteristics*** —risks associated with the sophistication of the customer and the developer's ability to communicate with the customer in a timely manner.
- ***Process definition*** —risks associated with the degree to which the software process has been defined and is followed by the development organization.
- ***Development environment*** —risks associated with the availability and quality of the tools to be used to build the product.
- ***Technology to be built*** —risks associated with the complexity of the system to be built and the "newness" of the technology that is packaged by the system.
- ***Staff size and experience*** —risks associated with the overall technical and project experience of the software engineers who will do the work.

Risk Components

PM identify the risk drivers that affect software risk components:

- *Performance risk*—the degree of uncertainty that the product will meet its requirements and be fit for its intended use.
- *Cost risk*—the degree of uncertainty that the project budget will be maintained.
- *Support risk*—the degree of uncertainty that the resultant software will be easy to correct, adapt, and enhance.
- *Schedule risk*—the degree of uncertainty that the project schedule will be maintained and that the product will be delivered on time.

The impact of each risk driver on the risk component

Components Category		Performance	Support	Cost	Schedule
Catastrophic	1	Failure to meet the requirement would result in mission failure		Failure results in increased costs and schedule delays with expected values in excess of \$500K	
	2	Significant degradation to nonachievement of technical performance	Nonresponsive or unsupportable software	Significant financial shortages, budget overrun likely	Unachievable IOC
Critical	1	Failure to meet the requirement would degrade system performance to a point where mission success is questionable		Failure results in operational delays and/or increased costs with expected value of \$100K to \$500K	
	2	Some reduction in technical performance	Minor delays in software modifications	Some shortage of financial resources, possible overruns	Possible slippage in IOC
Marginal	1	Failure to meet the requirement would result in degradation of secondary mission		Costs, impacts, and/or recoverable schedule slips with expected value of \$1K to \$100K	
	2	Minimal to small reduction in technical performance	Responsive software support	Sufficient financial resources	Realistic, achievable schedule
Negligible	1	Failure to meet the requirement would create inconvenience or nonoperational impact		Error results in minor cost and/or schedule impact with expected value of less than \$1K	
	2	No reduction in technical performance	Easily supportable software	Possible budget underrun	Early achievable IOC

Risk Mitigation, Monitoring, and Management

- Risk analysis goal - to assist the project team in developing a strategy for dealing with risk. An effective strategy must consider three issues:
 - Risk avoidance or mitigation.
 - Risk monitoring
 - Risk management and contingency planning
- Proactive approach to risk, avoidance is always the best strategy. This is achieved by developing a plan for *risk mitigation*.
- For example, assume that high staff turnover (i.e. revenue) is noted as a project risk.

- To mitigate this risk, project management must develop a strategy for reducing turnover.
- Steps are:
 - Meet with current staff to determine causes for turnover (e.g., low pay, competitive job market).
 - Mitigate those causes that are under our control before the project starts.
 - Organize project teams so that information about each development activity is widely dispersed.
 - Define documentation standards and establish mechanisms to be sure that documents are developed in a timely manner.
 - Conduct peer reviews of all work
 - Assign a backup staff member for every critical technologist.

- As the project proceeds, *risk monitoring* activities commence.
- The project manager monitors factors that may provide an indication of whether the risk is becoming more or less likely.
- In the case of high staff turnover, the following factors can be monitored:
 - General attitude of team members based on project pressures.
 - Potential problems with compensation and benefits.
 - The availability of jobs within the company and outside it.
- *Risk management and contingency planning* assumes that mitigation efforts have failed and that the risk has become a reality.

- The project is well underway and a number of people announce that they will be leaving. If the mitigation strategy has been followed, backup is available, information is documented, and knowledge has been dispersed across the team.
- In addition, the project manager may temporarily refocus resources (and readjust the project schedule) to those functions that are fully staffed, enabling newcomers who must be added to the team to “get up to speed.”
- Those individuals who are leaving are asked to stop all work and spend their last weeks in “knowledge transfer mode.”
- This might include video-based knowledge capture, the development of “commentary documents,” and/or meeting with other team members who will remain on the project.

RMMM steps incur additional project cost. For example, spending the time to “backup” every critical technologist costs money.

THE RMMM PLAN

- The RMMM plan documents all work performed as part of risk analysis and is used by the project manager as part of the overall project plan.
- Some software teams do not develop a formal RMMM document. Rather, each risk is documented individually using a *risk information sheet* (RIS).
- RIS is maintained using a database system, so that creation and information entry, priority ordering, searches, and other analysis may be accomplished easily.
- Once RMMM has been documented and the project has begun, risk mitigation and monitoring steps commence.

Risk information sheet

Risk ID: P02-4-32

Date: 5/9/02

Prob: 80%

Impact: high

Description:

Only 70 percent of the software components scheduled for reuse will, in fact, be integrated into the application. The remaining functionality will have to be custom developed.

Refinement/context:

Subcondition 1: Certain reusable components were developed by a third party with no knowledge of internal design standards.

Subcondition 2: The design standard for component interfaces has not been solidified and may not conform to certain existing reusable components.

Subcondition 3: Certain reusable components have been implemented in a language that is not supported on the target environment.

Mitigation/monitoring:

1. Contact third party to determine conformance with design standards.
2. Press for interface standards completion; consider component structure when deciding on interface protocol.
3. Check to determine number of components in subcondition 3 category; check to determine if language support can be acquired.

Management/contingency plan/trigger:

RE computed to be \$20,200. Allocate this amount within project contingency cost. Develop revised schedule assuming that 18 additional components will have to be custom built; allocate staff accordingly.

Trigger: Mitigation steps unproductive as of 7/1/02

Current status:

5/12/02: Mitigation steps initiated.

Originator: D. Gagne

Assigned: B. Laster

Software Quality Assurance

Quality Concepts

1. Quality

- Quality as “a characteristic or attribute of something.”
- Two kinds of quality may be encountered:
 - *Quality of design* of a product increases, if the product is manufactured according to specifications.
 - *Quality of conformance* is the degree to which the design specifications are followed during manufacturing.
- In software development,
 - Quality of design encompasses requirements, specifications, and the design of the system.
 - Quality of conformance is an issue focused primarily on implementation.

User satisfaction = compliant product + good quality + delivery within budget and schedule

Software Quality Assurance (SQA)

- Definition:
 - Conformance to explicitly stated functional and performance requirements, explicitly documented development standards, and implicit characteristics that are expected of all professionally developed software.
- Definition serves to emphasize three important points:
 - Software requirements are the **foundation** from which quality is measured. Lack of conformance to requirements is lack of quality.
 - Specified **standards** define a set of development criteria, If the criteria are not followed, lack of quality will almost surely result.
 - A set of **implicit requirements** often goes unmentioned. If software conforms to its explicit requirements but fails to meet implicit requirements, software quality is suspect

SQA group activities

- SQA group made up of software engineers, project managers, customers, salespeople, and the individuals members.
- Software quality assurance is composed of two different constituencies.
 - Software engineers who do technical work
 - SQA group that has responsibility for quality assurance planning, oversight, record keeping, analysis, and reporting.
- Software engineers address quality activities by applying technical methods and measures, conducting formal technical reviews, and performing well-planned software testing.
- SQA group is to assist the software team in achieving a high quality end product.

Role of an SQA group

1. Prepares an SQA plan for a project.

- The plan is developed during project planning and is reviewed by all stakeholders.
- The plan identifies
 - Evaluations to be performed
 - Audits and reviews to be performed
 - Standards that are applicable to the project
 - Procedures for error reporting and tracking
 - Documents to be produced by the SQA group
 - Amount of feedback provided to the software project team

- 2. Participates in the development of the project's software process description.**
 - The SQA group reviews the process description for compliance with organizational policy, internal software standards, externally imposed standards (e.g., ISO-9001), and other parts of the software project plan.
- 3. Reviews software engineering activities to verify compliance with the defined software process.**
 - The SQA group identifies, documents, and tracks deviations from the process and verifies that corrections have been made.
- 4. Audits designated software work products to verify compliance with those defined as part of the software process.**
 - The SQA group reviews selected work products; identifies, documents, and tracks deviations; verifies that corrections have been made; and periodically reports the results of its work to the project manager.

5. Ensures that deviations in software work and work products are documented and handled according to a documented procedure.

- Deviations may be encountered in the project plan, process description, applicable standards, or technical work products.

6. Records any noncompliance and reports to senior management.

- Noncompliance items are tracked until they are resolved.

Software Review

- Software reviews are a "filter" for the software engineering process.
- That is, reviews are applied at various points during software development and serve to uncover errors and defects that can then be removed.
- Types of Review
 - Informal Review
 - Meeting around the coffee machine and discussing technical problems.
 - Formal Review
 - Formal presentation of software design to an audience of customers, management, and technical staff
- A FTR is the most effective filter from a quality assurance standpoint.

Cost Impact of Software Defects

- The primary objective of formal technical reviews is to find errors during the process so that they do not become defects after release of the software.
- Direct benefit is early discovery of error, do not propagate to the next step.
- Design activities introduce between 50 and 65 percent of all errors (and ultimately, all defects) during the software process.
- However, FTR have been shown to be up to 75 percent effective in uncovering design flaws.
- FTP substantially reduces the cost of subsequent steps in the development and support phases.

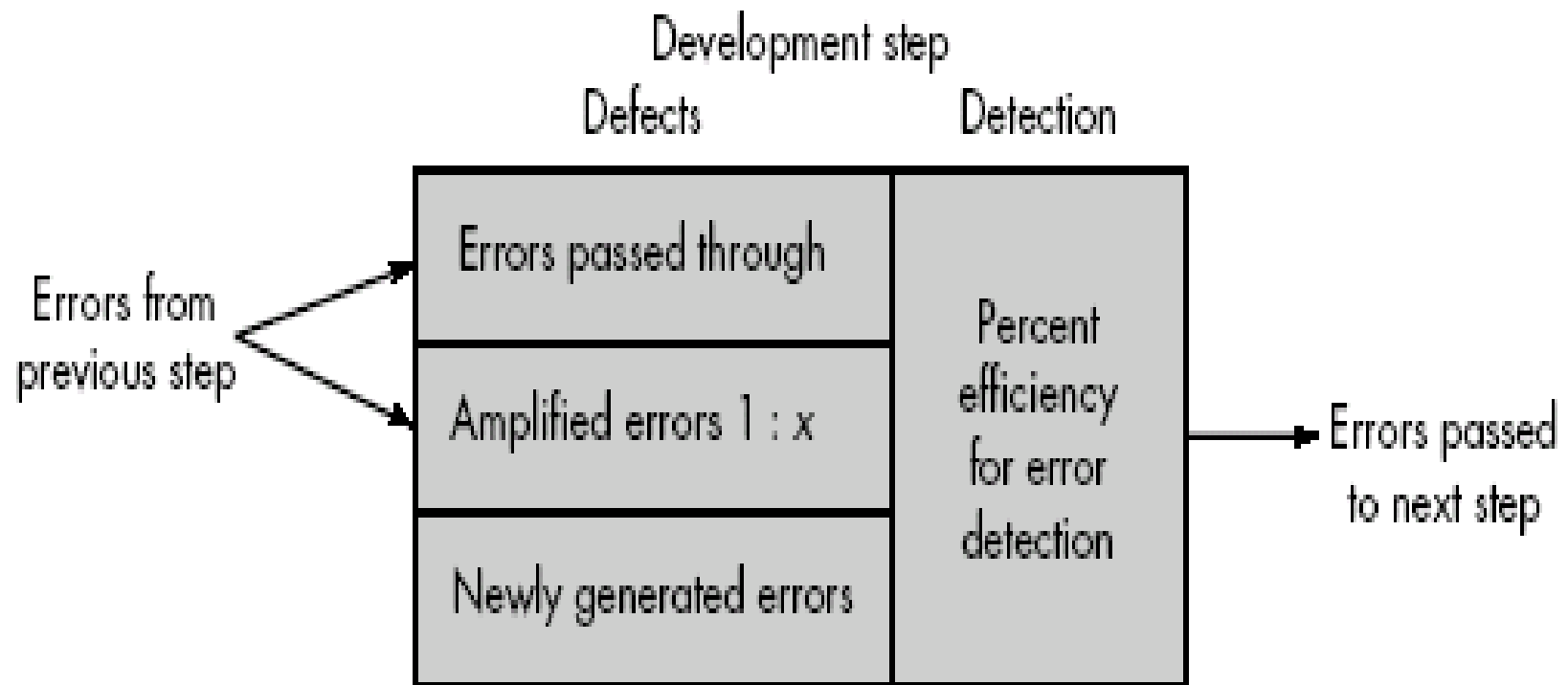
Contd.

- Assume that an error uncovered during design will cost 1.0 monetary unit to correct.
- Relative to this cost, the same error uncovered just before testing commences will cost 6.5 units; during testing, 15 units; and after release, between 60 and 100 units.

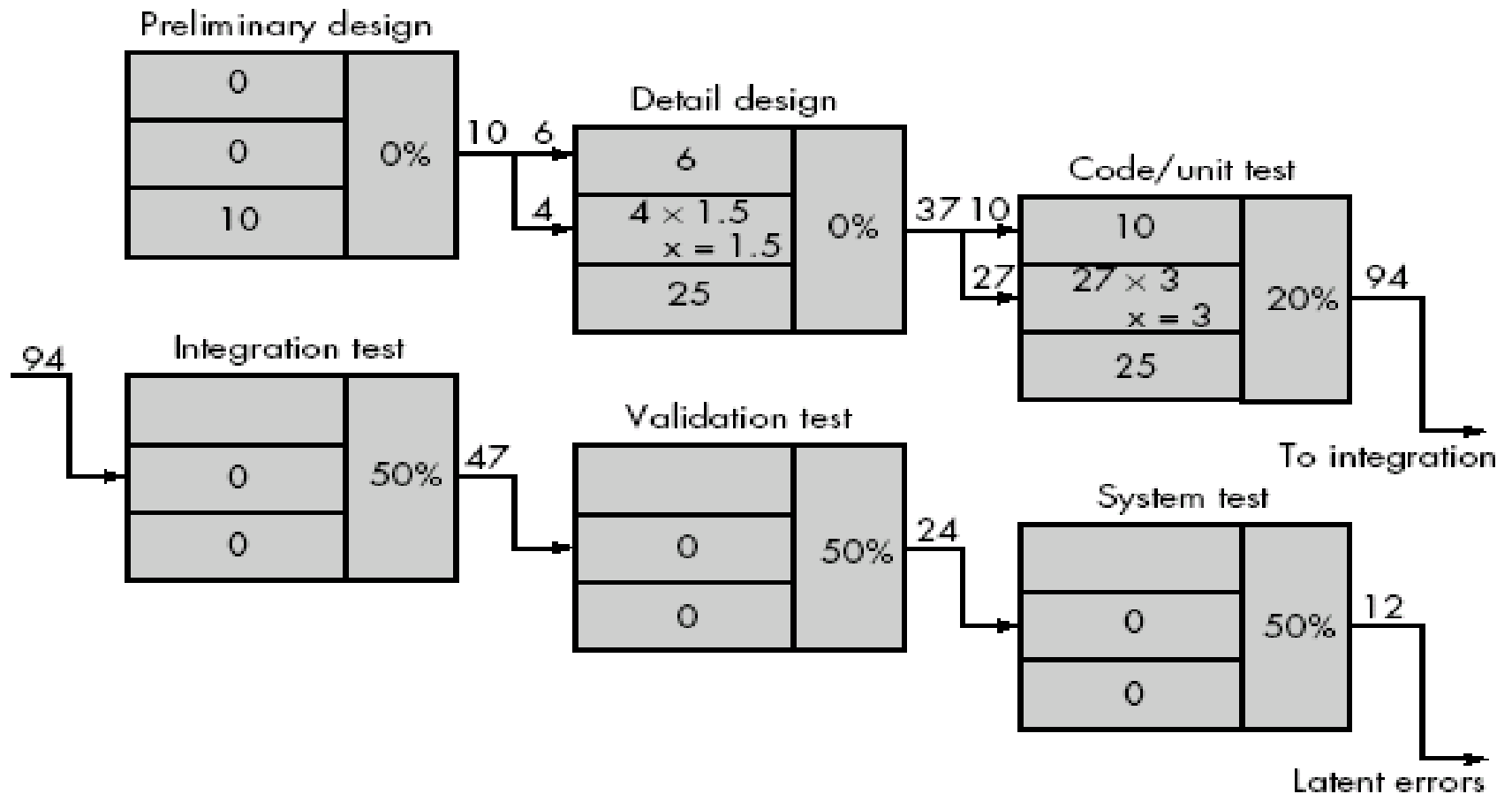
Defect Amplification and Removal

- A defect amplification model can be used to illustrate the generation and detection of errors during the preliminary design, detail design, and coding steps of the software engineering process.
- A box represents a software development step. During the step, errors may be by mistake generated.
- Review may fail to uncover newly generated errors and errors from previous steps, resulting in some number of errors that are passed through.
- In some cases, errors passed through from previous steps are amplified (amplification factor, x) by current work.
- The box subdivisions represent each of these characteristics and the percent of efficiency for detecting errors.

Defect Amplification model

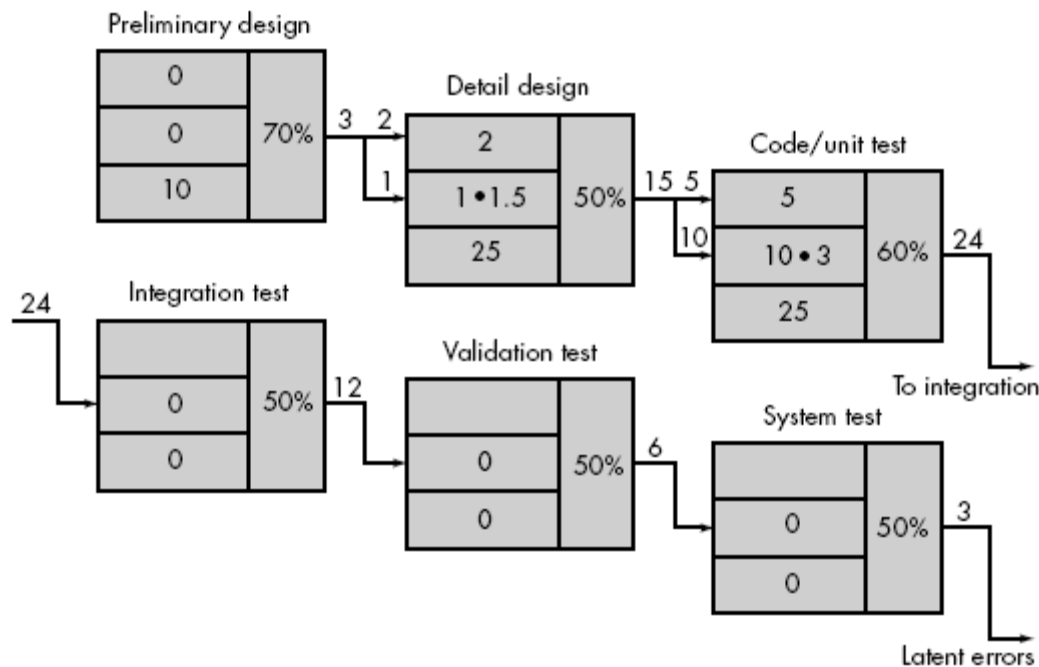


Defect Amplification – No Review



- Because of no review conducted, Ten preliminary design defects are amplified to 94 errors before testing commences.
- Each test step is assumed to correct 50 percent of all incoming errors without introducing any new errors.
- Twelve latent errors are released to the field or to the customers.

Defect Amplification – Review conducted



- Considers the same conditions except that design and code reviews are conducted as part of each development step.
- In this case, ten initial preliminary design errors are amplified to 24 errors before testing commences.
- Only three latent errors exist.
- Now we can estimate overall cost (with and without review for our hypothetical example)
- Using these data, the total cost for development and maintenance when reviews are conducted.
- To conduct reviews, a software engineer must expend time and effort and the development organization must spend money.

FORMAL TECHNICAL REVIEWS

- It is a software quality assurance activity performed by software engineers

Objectives of the FTR are

- To uncover errors in function, logic, or implementation for any representation of the software;
 - To verify that the software under review meets its requirements;
 - To ensure that the software has been represented according to predefined standards;
 - To achieve software that is developed in a uniform manner;
 - To make projects more manageable.
- The FTR is actually a class of reviews that includes walkthroughs, inspections, round-robin reviews and other small group technical assessments of software.

FTR- Review meeting

- Every review meeting should abide by the following constraints:
- Between three and five people (typically) should be involved in the review.
- Advance preparation should occur but should require no more than two hours of work for each person.
- The duration of the review meeting should be less than two hours.
- FTR focuses on a specific (and small) part of the overall software.
- e.g. rather than attempting to review an entire design, are conducted for each component or small group of components.

- The focus of the FTR is on a work product. (e.g. requirement, analysis, design, coding)
- *The producer*—informs the project leader that the work product is complete and that a review is required.
- The project leader contacts a *review leader*, who evaluates the product for readiness, generates copies of product materials, and distributes them to two or three reviewers for advance preparation.
- Each reviewer is expected to spend between one and two hours reviewing the product & making notes.
- Concurrently, the review leader also reviews the product and establishes an agenda for the review meeting, which is typically scheduled for the next day.
- The FTR begins with an introduction of the agenda and a brief introduction by the producer.
- The producer then proceeds to "walk through" the work product, explaining the material, while reviewers raise issues based on their advance preparation.

Contd.

- When valid problems or errors are discovered, the recorder notes each.
- At the end of the review, all attendees of the FTR must decide whether to
 - Accept the product without further modification,
 - Reject the product due to severe errors
 - Accept the product provisionally
- The decision made, all FTR attendees complete a sign-off, indicating their participation in the review and their concurrence with the review team's findings.

Review Reporting and Record Keeping

- During the FTR, a reviewer (the recorder) actively records all issues that have been raised.
- These are summarized at the end of the review meeting and a review issues list is produced.
- *A review summary report* answers three questions:
 1. What was reviewed?
 2. Who reviewed it?
 3. What were the findings and conclusions?
- *Review summary report* becomes part of the project historical record and may be distributed to the project leader and other interested parties.

- The *review issues list* serves two purposes:
 - To identify problem areas within the product
 - To serve as an action item checklist that guides the producer as corrections are made.
- An issues list is normally attached to the summary report.
- It is important to establish a follow-up procedure to ensure that items on the issues list have been properly corrected. Unless this is done, it is possible that issues raised can “fall between the cracks.”
- One approach is to assign the responsibility for follow-up to the review leader.

Review Guidelines

- *Review the product, not the producer.*
 - *Don't point out errors harshly. One way to be gentle is to ask a question that enables the producer to discover his or her own error.*
- *Set an agenda and maintain it.*
 - An FTR must be kept on track and on schedule.
- *Limit debate and rebuttal:*
 - Rather than spending time debating the question, the issue should be recorded for further discussion off-line
- *Enunciate problem areas, but don't attempt to solve every problem noted.*
 - Review only some small part of component.
- *Take written notes.*
 - make notes on a wall board, so that wording and priorities can be assessed by other reviewers

- *Limit the number of participants and insist upon advance preparation.*
 - Keep the number of people involved to the necessary minimum. However, all review team members must prepare in advance.
- *Develop a checklist for each product that is likely to be reviewed.*
 - helps the review leader to structure the FTR meeting and helps each reviewer to focus on important issues.
- *Allocate resources and schedule time for FTRs*
- *Conduct meaningful training for all reviewers.*
 - To be effective all review participants should receive some formal training
- *Review your early reviews.*

SOFTWARE RELIABILITY

- *Software reliability* is defined in statistical terms as "the probability of failure-free operation of a computer program in a specified environment for a specified time".
- What is meant by the term *failure*?
 - In the context of any discussion of software quality and reliability, failure is nonconformance to software requirements.
 - Correction of one failure may in fact result in the introduction of other errors that ultimately result in other failures.
- Software reliability can be measured directly and estimated using historical and developmental data.

Measures of Reliability and Availability

- A simple measure of reliability is *mean-time-between-failure* (MTBF), where

$$\text{MTBF} = \text{MTTF} + \text{MTTR}$$

The acronyms MTTF and MTTR are mean-time-to-failure and mean-time-to-repair, respectively.

- MTBF is a far more useful measure than defects/KLOC or defects/FP.
- Stated simply, an end-user is concerned with failures, not with the total error count. Because each error contained within a program does not have the same failure rate, the total error count provides little indication of the reliability of a system.
- In addition to a reliability measure, we must develop a measure of availability.

- *Software availability* is the probability that a program is operating according to requirements at a given point in time and is defined as

$$\text{Availability} = [\text{MTTF}/(\text{MTTF} + \text{MTTR})] \\ 100\%$$

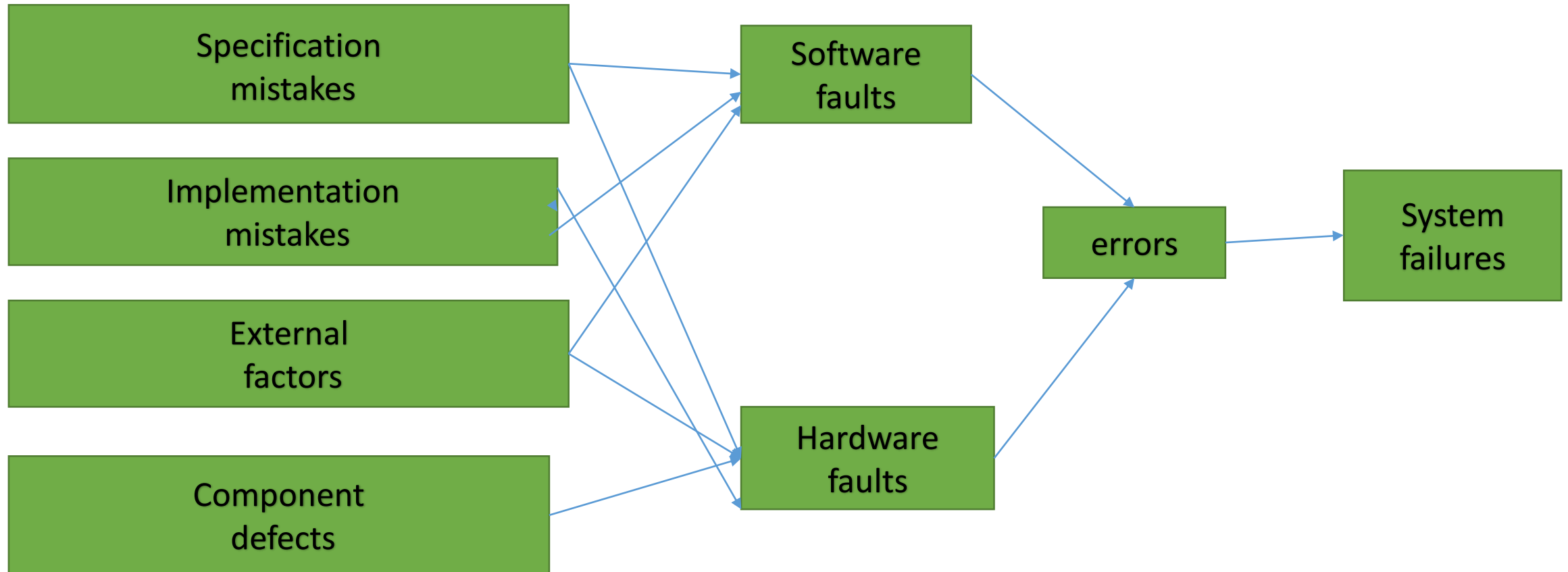
- The MTBF reliability measure is equally sensitive to MTTF and MTTR.
- The availability measure is somewhat more sensitive to MTTR, an indirect measure of the maintainability of software.

WHAT DO THEY MEAN??

➤ RELIABILITY

➤ SOFTWARE RELIABILITY

- Software reliability is based on the three primary concepts:
- Fault, error and failure (Bug in a program is fault, Possible incorrect values caused by this bug is an error. Possible crash of the OS is failure)

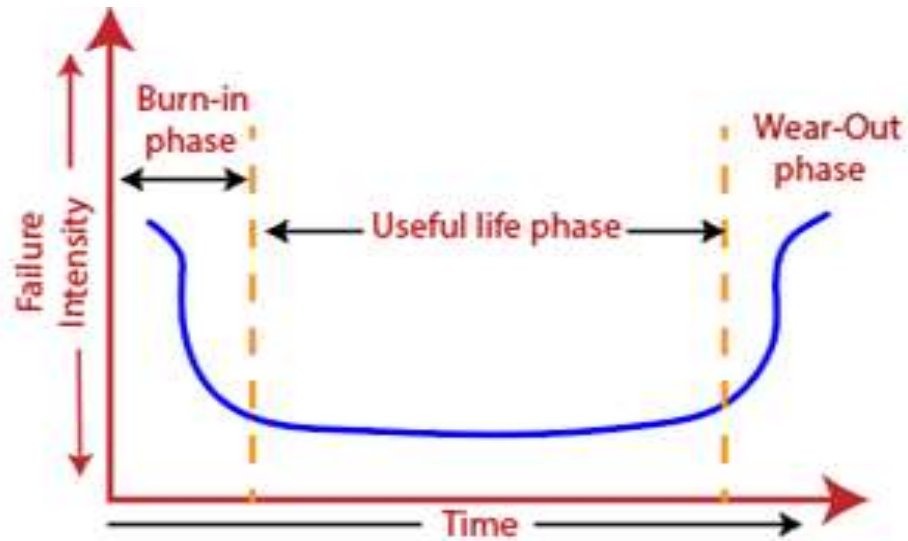


SOFTWARE FAILURE MECHANISMS:

- Transient failure
- Permanent failure
- Recoverable failure
- Unrecoverable failure
- Non-corruption failure
- Corrupting failure

HARDWARE RELIABILITY

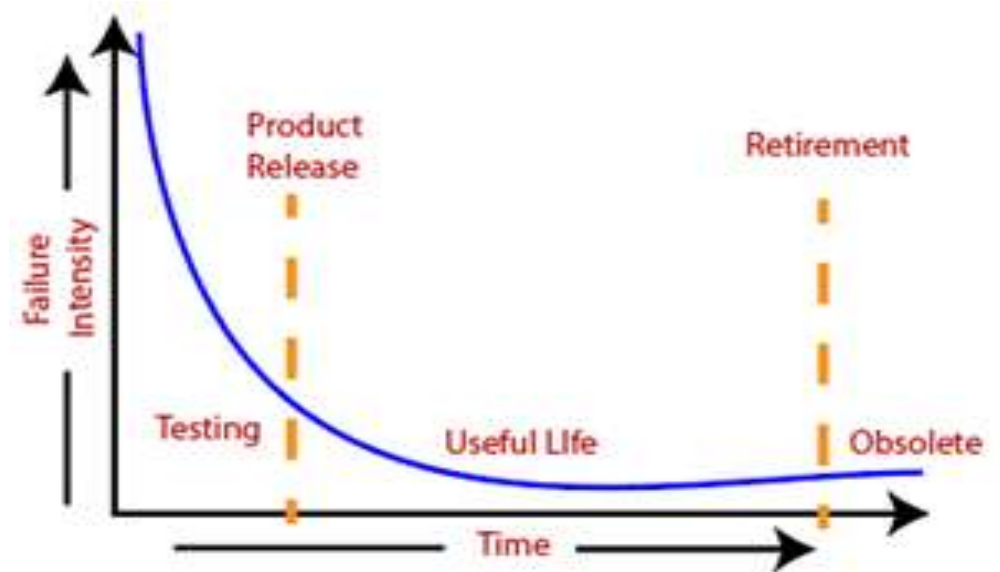
- PHYSICAL FAULTS
- Fail due to wear and t



(a) Hardware Product

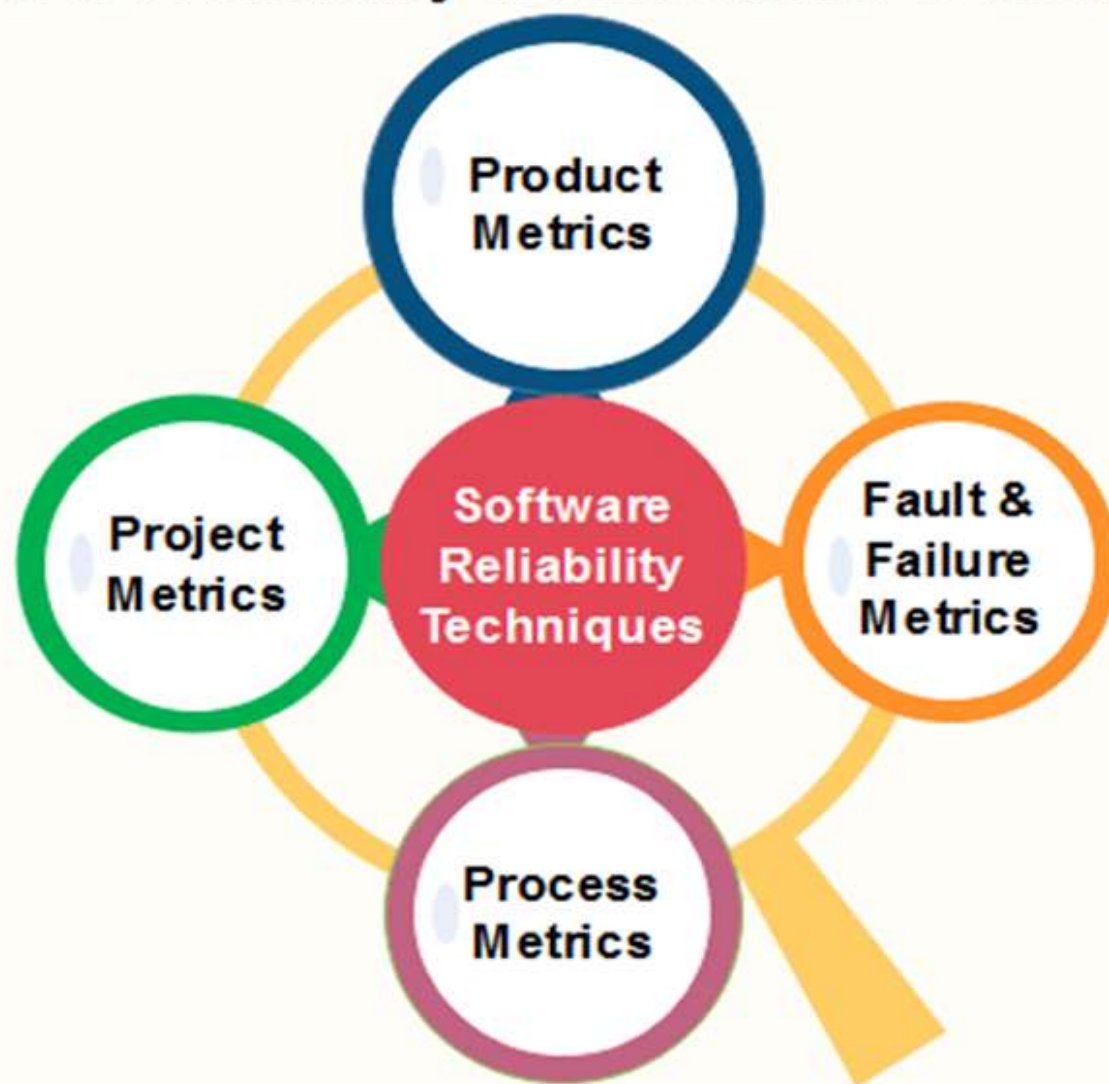
SOFTWARE RELIABILITY

- DESIGN FAULTS
- Fail due to bugs



(b) Software Product

Software Reliability Measurement Techniques



PRODUCT METRICS

- Product metrics are those which are used to **build the artifacts**.
- These metrics are:
 - i. Software size.
 - ii. Performance.
 - iii. Complexity.
 - iv. Defect Removal Efficiency(DRE).

PROJECT METRICS

- Project management metrics define **project characteristics and execution.**
- These metrics are:
 - i. Number of software developers.
 - ii. Cost and schedule.
 - iii. Productivity.

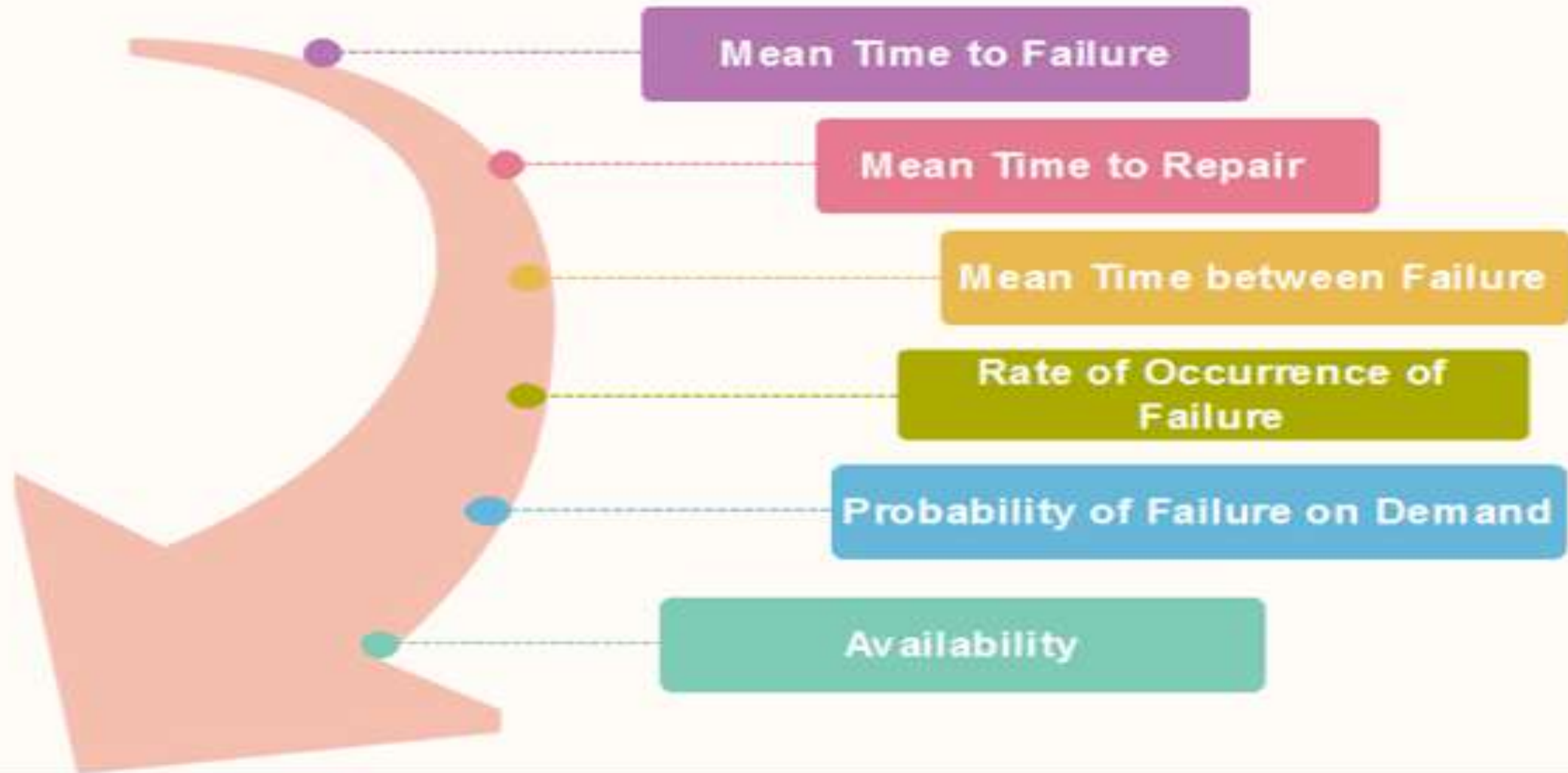
PROCESS METRICS

- Process metrics describe the **effectiveness and quality** of the processes that produce the software product.
- These metrics are :
 - i. Time.
 - ii. Efforts.
 - iii. Defects.

FAULT AND FAILURE METRICS

- A Fault is a defect in a program which appears when the programmer makes an error and causes failure when executed under particular conditions.
- These metrics are used to determine the failure-free execution software.
- Mean Time Between Failures(MTBF) or other parameters are used to measure or predict software reliability.

Reliability Metrics



1. Mean time to failure(MTTF):

$$\sum_{i=1}^n \frac{t_{i+1}-t_i}{(n-1)}$$

2. Mean time to repair(MTTR):

3. Mean time between failure(MTBF):

$$MTTF + MTTR$$

- An MTBF of 300

4. Rate of occurrence of failure (ROCOF):

- An ROCOF of 0.05

5. Probability of failure on demand (POFOD):

- Essential measure of safety-critical system
 - An POFOD of 0.8

6. Availability:

Probability that system is applicable for use at a given time

$$Availability = \frac{MTBF}{(MTBF + MTTR)} * 100$$

A signal processor software is expected to operate for 91.25 days after repair & s/w repair time is expected to be 5 mins. Availability of s/w is?

$$Availability = \frac{MTBF}{(MTBF + MTTR)} * 100$$

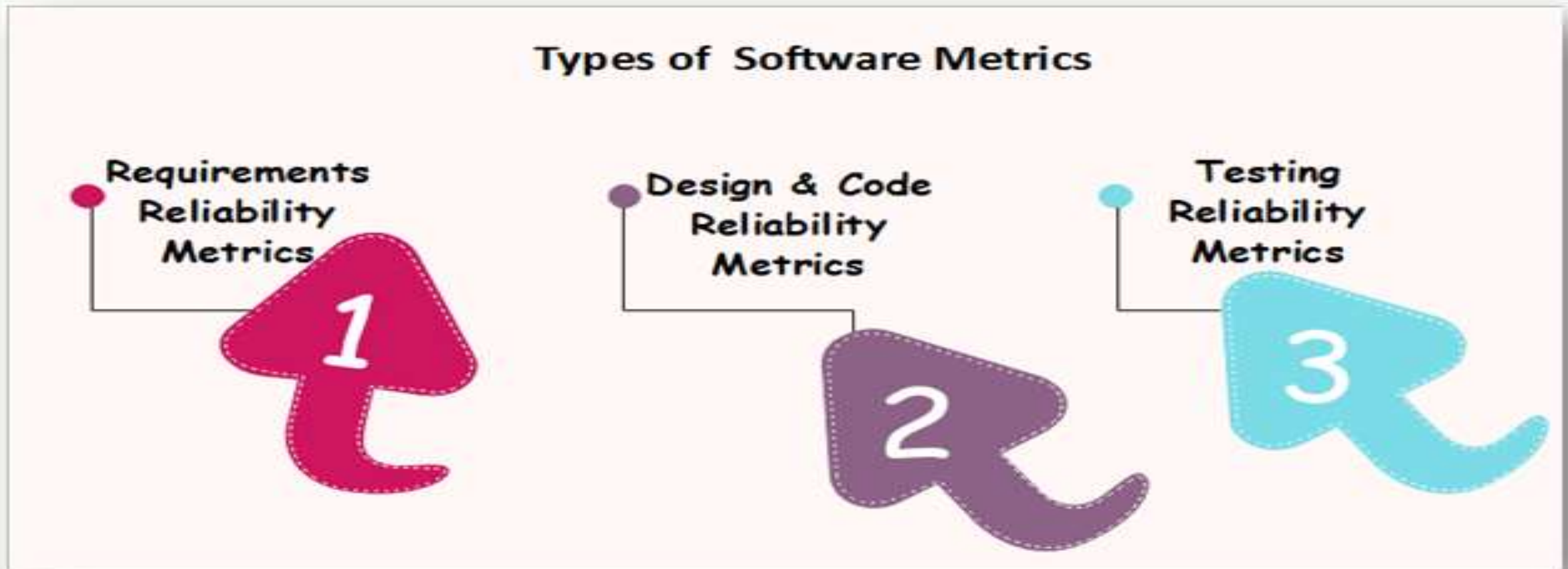
MTBF = 91.25 days => [91.25*24 hrs*60 min] mins = 131400 mins

MTTR = 5 mins

$$AVAILABILITY = \frac{131400}{131405} * 100 = 99.99629$$

Software Metrics for Reliability

The Metrics are used to improve the reliability of the system by identifying the areas of requirements.



REQUIREMENTS RELIABILITY METRICS:

- Features and Functionalities the software must include
- Avoiding miscommunication between developer and client
- Should not include inadequate data

DESIGN AND CODE RELIABILITY METRICS:

The quality methods that exists in design and coding plan are modularity ,complexity, size.

Complexity is inversely proportional to **Reliability**

TESTING RELIABILITY METRICS:

- It provides that the system is equipped with the tasks that are specified in the requirements. Because of this, the bugs due to the lack of functionality reduces.
- Calculating code, finding bugs, fixing them.
- GOAL???

SOFTWARE FAULT TOLERANCE

- ✓ Software faults are design faults
- ✓ Ability to detect and recover from fault
- ✓ Necessary for next generation computing systems

Following are the Software Fault Tolerance Techniques



Recovery Block



N-Version
Software



N-Version Software
and Recovery Block